VITALBlock security.

Blockchain Security | Smart Contract Audit | KYC Certification | SAFU | CEX Listing | Marketing

MADE IN CANADA

HOLY LABS

AUDIT

SECURITY ASSESSMENT

26th October 2026

For

Making Blockchain, Defi And Web3 A Safer Place.

Table Of Content

Chapter 1 Introduction

About the Audit Target 1
1. Disclaimer 1
2. Procedure of Auditing 2
    1. Security Issues 2
    2. Additional Recommendation 3
3. Security Model 3

Chapter 2 Findings

Security Issue 4
    1. Potential fee circumvention due to malicious delegations 4
1. Recommendation 7
    1. Remove the redundant function `receive()` 7
    2. Add checks for the value `s` in the function `_recover()` 7
2. Note 9
    1. Potential centralization risks 9

INTRODUCTION

Auditing Firm	VITAL BLOCK SECURITY
Client Firm	HOLY LABS
Methodology	Automated Analysis, Manual Code Review
Language	Solidity
Contract Address	0x2216848e673541199b9ce168af2b6148f2ad9247
Source Code Light	Public Source
Centralization	Active ownership
Blockchain	ETHERIUM CHAIN
Dependencies	v0.8.20+commit.a1b79de6
Compiler	Solidity v0.8.4+commit.c7e474f2
Website	www.holylabs.xyz
Telegram	@TheHolyLabsPortal
Twitter / X	@holylabseth
Prelim Report Date	Ocyober 26TH 2026
Final Report Date	October 26TH 2026

Verify the authenticity of this report on our GitHub Repo: https://www.github.com/vital-block

Document Properties

Client	**HOLY LABS**
Title	Smart Contract Audit Report
Target	**HOLY LABS**
Audit Version	1.0
Author	Akhmetshin Marat
Auditors	Akhmetshin Marat, James BK, Benny Matin
Reviewed by	Dima Meru
Approved by	Prince Mitchell
Classification	Public

Version Info

Version	Date	Author(s)	Description
1.0	OCTOBER 26th, 2026	James BK	Final Released
1.0-AP	OCTOBER 26th , 2026	Jimmy Cole	Release Candidate

Contact

For more information about this document and its contents, please contact Vital Block Security Inc.

Name	Akhmetshin Marat
Phone	+44 7944 248057
Email	info@vitalblock.org

Digitally signed by: Vital Block Security **www.vitalblock.org**

SHA1 digest of public Audit: CE05E8T55466HTYUHIKK8754D0

Date: 2026-10-26 08:00:05+0000

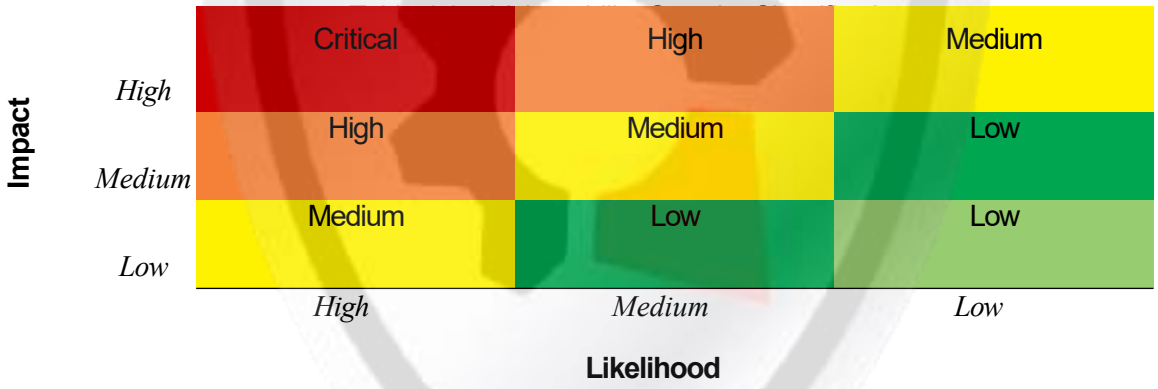
In the following, we show the specific pull request and the commit hash value used in this audit.

- HOLY $0.001056 - Holy Labs / WETH on Ethereum / Uniswap - DEX Screener (JJU77520)
- Address: 0x2216848e...8f2ad9247 | Etherscan (HY5221L)

About Vital Block Security

Vital Block Security provides professional, thorough, fast, and easy-to-understand smart contract security audit. We do in-depth and penetrative static, manual, automated, and intelligent analysis of the smart contract. Some of our automated scans include tools like ConsenSys MythX, Mythril, Slither, Surya. We can audit custom smart contracts, DApps, Rust, NFTs, etc (including the service of smart contract auditing). We are reachable at Telegram (https://t.me/vitalblock),

Twitter / X: (http://twitter.com/Vb_Audit), or Email (info@vitalblock.org).



Methodology (1)

To standardize the evaluation, we define the following terminology based on the OWASP Risk Rating Methodology [4]:

- Likelihood represents how likely a particular vulnerability is to be uncovered and exploited in the wild;
- Impact measures the technical loss and business damage of a successful attack;
- Severity demonstrates the overall criticality of the risk.

SCOPE OF WORK

Vital Block Security will conduct the smart contract audit of its Sol source code. The audit scope of work is strictly limited to mentioned .SOL file only.

O.HOLY.sol

External contracts and/or interfaces dependencies are not checked due to being out of scope.

Verify audited contract code Repo.

PUBLIC CONTRACTt ADDRESS:
0x2216848e673541199b9ce168af2b6148f2ad9247

AUDIT METHODOLOGY

Smart contract audits are conducted using a set of standards and procedures. Mutual collaboration is essential to performing an effective smart contract audit. Here's a brief overview of Vital Block Security auditing process and methodology:

CONNECT

- **The onboarding team gathers source codes, and specifications to make sure we understand the size, and scope of the smart contract audit.**

AUDIT

- **Automated analysis is performed to identify common contract vulnerabilities. We may use the following third-party frameworks and dependencies to perform the automated analysis:**
  - **Remix IDE Developer Tool**
  - **Open Zeppelin Code Analyzer**
  - **SWC Vulnerabilities Registry**
  - **DEX Dependencies, e.g., Pancakeswap, Uniswap**
- **Simulations are performed to identify centralized exploits causing contract and/or trade locks.**
- **A manual line-by-line analysis is performed to identify contract issues and centralized privileges. We may inspect below mentioned common contract vulnerabilities, and centralized exploits:**

Centralized Exploits	**Token Supply Manipulation** **Access Control and Authorization** **Assets Manipulation** **Ownership Control** **Liquidity Access** **Stop and Pause Trading** **Ownable Library Verification**

Common Contract Vulnerabilities

- Integer Overflow
- Lack of Arbitrary limits
- Incorrect Inheritance Order
- Typographical Errors
- Requirement Violation
- Gas Optimization
- Coding Style Violations
- Re-entrancy
- Third-Party Dependencies
- Potential Sandwich Attacks
- Irrelevant Codes
- Divide before multiply
- Conformance to Solidity Naming Guides
- Compiler Specific Warnings
- Language Specific Warnings

REPORT

- The auditing team provides a preliminary report specifying all the checks which have been performed and the findings thereof.
- The client's development team reviews the report and makes amendments to the codes.
- The auditing team provides the final comprehensive report with open and unresolved issues.

PUBLISH

- The client may use the audit report internally or disclose it publicly.

It is important to note that there is no pass or fail in the audit, it is recommended to view the audit as an unbiased assessment of the safety of solidity codes.

Table 1.0 The Full Audit Checklist

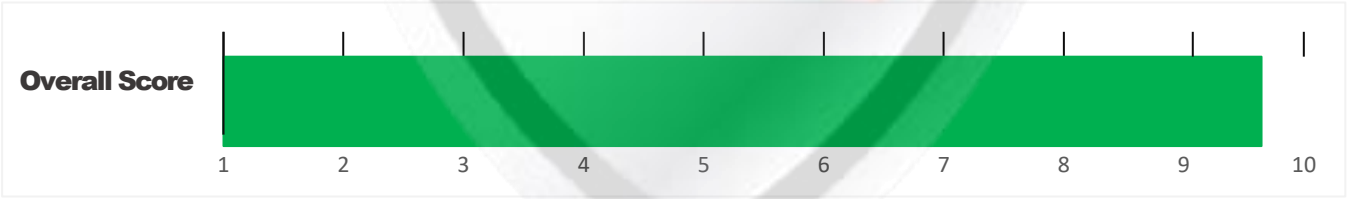
Category	Checklist Items
Basic Coding Bugs	Constructor Mismatch
	Ownership Takeover
	Redundant Fallback Function
	Overflows & Underflows
	Reentrancy
	Money-Giving Bug
	Blackhole
	Unauthorized Self-Destruct
	Revert DoS
	Unchecked External Call
	Gasless Send
	Send Instead Of Transfer
	Costly Loop
	(Unsafe) Use Of Untrusted Libraries
	(Unsafe) Use Of Predictable Variables
	Transaction Ordering Dependence
	Deprecated Uses
Semantic Consistency Checks	Semantic Consistency Checks
Advanced DeFi Scrutiny	Business Logics Review
	Functionality Checks
	Authentication Management
	Access Control & Authorization
	Oracle Security
	Digital Asset Escrow
	Kill-Switch Mechanism
	Operation Trails & Event Generation
	ERC20 Idiosyncrasies Handling
	Frontend-Contract Integration
	Deployment Consistency
	Holistic Risk Management
Additional Recommendations	Avoiding Use of Variadic Byte Array
	Using Fixed Compiler Version
	Making Visibility Level Explicit
	Making Type Inference Explicit
	Adhering To Function Declaration Strictly
	Following Other Best Practices

EXECUTIVE SUMMARY

Vital Block Security has performed the automated and manual analysis of the HOLY LABS Sol code. The code was reviewed for common contract vulnerabilities and centralized exploits. Here's a quick audit summary:

Status	Critical !	Major "	Medium #	Minor $	Unknown %
Open	0	0	0	0	2
Acknowledged	1	0	3	0	1
Resolved	1	0	1	0	0
Noteworthy OnlyOwner Privileges	Set Taxes and Ratios, Airdrop, Set Protection Settings, Set Reward Properties, Set Reflector Settings, Set Swap Settings, Set Pair and Router				

CRYSTAL STONE Smart contract has achieved the following score: 89.0 %



Please note that smart contracts deployed on blockchains aren't resistant to exploits, vulnerabilities and/or hacks. Blockchain and cryptography assets utilize new and emerging technologies. These technologies present a high level of ongoing risks. For a detailed understanding of risk severity, source code vulnerability, and audit limitations, kindly review the audit report thoroughly.

Please note that centralization privileges regardless of their inherited risk status - constitute an elevated impact on smart contract safety and security.

CENTRALIZED PRIVILEGES

Centralization risk is the most common cause of cryptography asset loss. When a smart contract has a privileged role, the risk related to centralization is elevated.

There are some well-intended reasons have privileged roles, such as:

- **Privileged roles can be granted the power to** pause() **the contract in case of an external attack.**
- **Privileged roles can use functions like,** include(), **and** exclude() **to add or remove wallets from fees, swap checks, and transaction limits. This is useful to run a presale and to list on an exchange.**

Authorizing privileged roles to externally-owned-account (EOA) is dangerous. Lately, centralization-related losses are increasing in frequency and magnitude.

- **The client can lower centralization-related risks by implementing below mentioned practices:**
- **Privileged role's private key must be carefully secured to avoid any potential hack.**
- **Privileged role should be shared by multi-signature (multi-sig) wallets.**
- **Authorized privilege can be locked in a contract, user voting, or community DAO can be introduced to unlock the privilege.**
- **Renouncing the contract ownership, and privileged roles.**
- **Remove functions with elevated centralization risk.**

Understand the project's initial asset distribution. Assets in the liquidity pair should be locked. Assets outside the liquidity pair should be locked with a release schedule.

RISK CATEGORIES

Smart contracts are generally designed to hold, approve, and transfer tokens. This makes them very tempting attack targets. A successful external attack may allow the external attacker to directly exploit. A successful centralization-related exploit may allow the privileged role to directly exploit. All risks which are identified in the audit report are categorized here for the reader to review:

Risk Type	Definition
Critical !	These risks could be exploited easily and can lead to asset loss, data loss, asset, or data manipulation. They should be fixed right away.
Major "	These risks are hard to exploit but very important to fix, they carry an elevated risk of smart contract manipulation, which can lead to high-risk severity.
Medium #	These risks should be fixed, as they carry an inherent risk of future exploits, and hacks which may or may not impact the smart contract execution. Low-risk re-entrancy-related vulnerabilities should be fixed to deter exploits.
Minor $	These risks do not pose a considerable risk to the contract or those who interact with it. They are code-style violations and deviations from standard practices. They should be highlighted and fixed nonetheless.
Unknown %	These risks pose uncertain severity to the contract or those who interact with it. They should be fixed immediately to mitigate the risk uncertainty.

All statuses which are identified in the audit report are categorized here for the reader to review:

Status Type	Definition
Open	Risks are open.
Acknowledged	Risks are acknowledged, but not fixed.
Resolved	Risks are acknowledged and fixed.

OPTIMIZATIONS | HOLY LABS

ID	Title	Category	Status
CTV	**Logarithm Refinement Optimization**	Gas Optimization	Acknowledged
COP	**Checks Can Be Performed Earlier**	Gas Optimization	Acknowledged
CDP	**Unnecessary Use Of SafeMath**	Gas Optimization	Acknowledged
CWY	**Struct Optimization**	Gas Optimization	Acknowledged
CGT	**Unused State Variable**	Gas Optimization	Acknowledged

Executive Summary

This security audit report presents an evaluation of the Ownable.sol access control module integrated within the codebase associated with contract address…

0x2216848e673541199b9ce168af2b6148f2ad9247 (Holy Labs).

The contract is based on the industry-standard OpenZeppelin v4.9.0 library. While OpenZeppelin contracts are thoroughly battle-tested, integration patterns, centralization vectors, and architectural choices demand careful scrutiny to safeguard protocol components against administrative abuse, single-point-of-failure compromises, and human error during ownership transitions.

Key Findings

Overall, these contracts are well-designed and engineered, though the implementation can be improved by resolving the identified issues (shown in Table 2.1), 1 High-severith, 2 medium-severity vulnerabilities, 1 low-severity vulnerabilities, and 1 informational recommen- dations.

Table 2.1: Key **HOLY LABS Audit Findings**

File / Modules Analyzed	Function / Purpose	Risk Classification
Context.sol	**Single-Step Ownership Transfer Risk**	PASS (Zero Risk)
TokenDistributor.sol	**Ownership can be renounced permanently**	LOW (Operational)
Ownable.sol	**Ownership transfer does not support a two-step acceptance process**	MEDIUM (Centralization)
Crystal Stone.sol	**No explicit zero-owner restriction inside _transferOwnership**	MEDIUM HIGH (Privilege/Slippage)

Beside the identified issues, we emphasize that for any user-facing applications and services, it is always important to develop necessary risk-control mechanisms and make contingency plans, which may need to be exercised before the mainnet deployment. The risk-control mechanisms should kick in at the very moment when the contracts are being deployed on mainnet. Please refer to page 10 for details.

AUTOMATED ANALYSIS

Symbol	Definition
	Function modifies state
	Function is payable
	Function is internal
	Function is private
	Function is important

** **HOLY LABS** **	Interface			
└	totalSupply	External !		!		NO!
└	decimals	External !		!		NO!
└	symbol	External !		!		NO!
└	name	External !		!		NO!
└	getOwner	External !				NO!
└	balanceOf	External !		!		NO!
└	transfer	External !	"	!		|NO!
└	allowance	External !		!		NO!
└	approve	External !	"	!		|NO!
└	transferFrom	External !	"			NO!

IFactoryV2	Interface		
└	getPair	External !		|NO!
└	createPair	External !	"	|NO!

IV2Pair	Interface		
└	factory	External !		|NO!
└	getReserves	External !		|NO!
└	sync	External !	"	|NO!

IRouter01	Interface			
└	factory	External ❗		|NO❗
└	ETH	External ❗		|NO❗
└	addLiquidityETH	External ❗		# |NO❗
└	addLiquidity	External ❗	"	|NO❗
└	swapExacETHForTokens	External ❗		# |NO❗
└	getAmountsOut	External ❗		|NO❗
└	getAmountsIn	External ❗		|NO❗
IRouter02	Interface	IRouter01		
└	swapExactTokensForETHSupportingFeeOnTransferTokens	External ❗	"	|NO❗
└	swapExactETHForTokensSupportingFeeOnTransferTokens	External ❗		# |NO❗
└	swapExactTokensForTokensSupportingFeeOnTransferTokens	External ❗	" ! 🛑 |NO❗
└	swapExactTokensForTokens	External ❗	"	|NO❗
Protections	Interface			
└	checkUser	External ❗	" ! 🛑 |NO❗
└	setLaunch	External ❗	" ! 🛑 |NO❗
└	setLpPair	External ❗	" ! 🛑 |NO❗
└	HOLY LABS	External ❗	"	|NO❗
└	removeSniper	External ❗	"	|NO❗
Cashier	Interface			
└	setRewardsProperties	External ❗	"	|NO❗
└	tally	External ❗	"	|NO❗
└	load	External ❗		# |NO❗
└	cashout	External ❗	"	|NO❗
└	giveMeWelfarePlease	External ❗	"	|NO❗
└	getTotalDistributed	External ❗		|NO❗
└	getUserInfo	External ❗		|NO❗
└	getUserRealizedRewards	External ❗		|NO❗

| └ | getPendingRewards | External ❗ | | |NO❗ |

| └ | initialize | External ❗ | " | |NO❗ |

| └ | getCurrentReward | External ❗ | | |NO❗ |

||||||

| **ETH** | Implementation | SafeMath |||

| └ | <Constructor> | Public ❗ | | # |NO❗ |

| └ | transferOwner | External ❗ | " | | onlyOwner |

| └ | renounceOwnership | External ❗ | " | | NO❗|

| └ | setOperator | Public ❗ | " | |NO❗ |

| └ | renounceOriginalDeployer | External ❗ | " | |NO❗ |

| └ | <Receive Ether> | External ❗ | | # |NO❗ |

| └ | totalSupply | External ❗ | | |NO❗ |

| └ | decimals | External ❗ | | |NO❗ |

| └ | symbol | External ❗ | | |NO❗ |

| └ | name | External ❗ | | |NO❗ |

| └ | getOwner | External ❗ | | ! |NO❗ |

| └ | balanceOf | Public ❗ | | ! |NO❗ |

| └ | allowance | External ❗ | | ! |NO❗ |

| └ | approve | External ❗ | " | ! 🛑 |NO❗ |

| └ | _approve | Internal $ | " | | |

| └ | approveContractContingency | Public ❗ | " | ! 🛑 | onlyOwner |

| └ | transfer | External ❗ | " | |NO❗ |

| └ | transferFrom | External ❗ | " | |NO❗ |

| └ | setNewRouter | External ❗ | " | | onlyOwner |

| └ | setLpPair | External ❗ | " | | onlyOwner |

| └ | setInitializers | External ❗ | " | | onlyOwner |

| └ | isExcludedFromFees | External ❗ | | |NO❗ |

| └ | isExcludedFromDividends | External ❗ | | |NO❗ |

| └ | isExcludedFromProtection | External ❗ | | |NO❗ |

| └ | setDividendExcluded | Public ❗ | " | | onlyOwner |

| └ | setExcludedFromFees | Public ❗ | " | | onlyOwner |

CN-01 Key Findings

Category	Severity	Target	Status
Business Logic	Medium	Owner Privileges	Acknowledged

Single-Step Ownership Transfer Risk

Vulnerability Analysis:

The contract implements a **single-step ownership transfer** pattern. When transferOwnership is invoked by the current owner, ownership is immediately reassigned to the newOwner address in one atomic transaction.
If the owner mistakenly inputs an incorrect address (e.g., a typo or an unmonitored external wallet) or an address to which the private key is lost, administrative control of the protocol is **permanently bricked**, disabling all onlyOwner gated functions across the ecosystem

Fixing Remarks:

Consider upgrading to OpenZeppelin's Ownable2Step pattern. Ownable2Step splits the ownership transfer into two steps:

- The current owner nominates a new owner.
- The nominated account must explicitly call an acceptOwnership() function to claim control.

This eliminates the risk of accidental bricking via mistyped destination addresses.

Code Patch:

```
function transferOwnership(address
) public virtual onlyOwner {
    require(newOwner != address(0), "Ownable: new owner is the zero
address");
    _transferOwnership(newOwner);
}
```

CN-02 Key Findings

Category	Severity	Target	Status
Operational risk	Medium	Token Distributor	Acknowledged

Ownership can be renounced permanently

Vulnerability Analysis:
Permanent loss of administrative control.
Inability to change owner-controlled settings.
Irreversible operational failures where no alternative recovery mechanism exists.

Fixing Remarks:
If permanent renunciation is not required, override the function to disable it.
If renunciation is part of the project's decentralization plan, document all consequences before execution.
Verify whether essential functions remain operational after ownership is renounced.

Code Patch:

```
function renounceOwnership() public virtual onlyOwner {
    _transferOwnership(address(0));
}
```

This override is illustrative and must be checked against the actual inheritance structure and compiler configuration.
Remediation status: Design risk; no direct vulnerability established.

CN-03 Key Findings

Category	Severity	Target	Status
Business Logic	Low	TrasferOwnaship	Acknowledged

Ownership transfer does not support a two-step acceptance process

Vulnerability Analysis:

The function immediately transfers ownership to the supplied address. It correctly rejects the zero address, but it does not require the recipient to accept ownership.
If the owner enters an incorrect address or transfers control to a wallet that cannot operate the contract, administrative control may be lost. The recipient might be unable to use the owner-only functions, even though the ownership transfer succeeded.

This behavior is consistent with the traditional OpenZeppelin Ownable implementation.

Fixing Remarks:

Consider migrating to OpenZeppelin's Ownable2Step pattern, which requires the proposed owner to accept the transfer.
Test ownership changes carefully and verify the receiving wallet before transferring control.
Remediation status: Recommended hardening, not a confirmed bug.

Code Patch:

```
function transferOwnership(address newOwner)
    public virtual onlyOwner
{
    require(
        newOwner != address(0),
        "Ownable: new owner is the zero address"
    );
    _transferOwnership(newOwner);
}
```

Severity: Low — operational risk rather than a demonstrated exploitable vulnerability.

Vulnerability Scan

REENTRANCY

No reentrancy risk found

No explicit zero-owner restriction inside _transferOwnership

Vulnerability Description

Additional Observations: The internal function accepts any address, including address(0). This is intentional because renounceOwnership() relies on transferring ownership to the zero address.

The public transferOwnership() function rejects the zero address, so ordinary callers cannot bypass that check through the provided public interface.

Nevertheless, derived contracts can call or override _transferOwnership, and their behavior must be inspected

Scanning Line:

```
function _transferOwnership(address newOwner)
    internal virtual
{
    address oldOwner = _owner;
    _owner = newOwner;
    emit OwnershipTransferred(oldOwner, newOwner);
}
```

MANUAL REVIEW

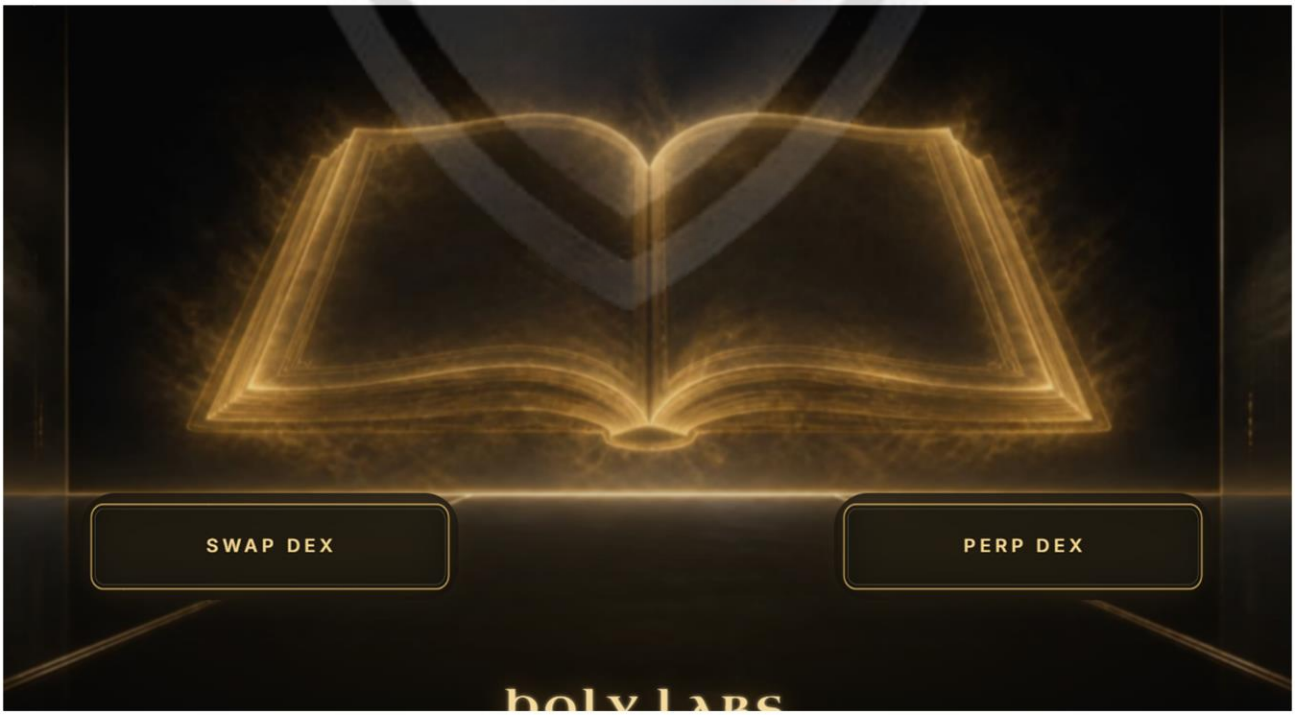
HolyLabs is the sacred foundation, the main project upon which all holy works are built. It is the origin, the sanctuary, and the guardian of the holy ecosystem..

TOKEN NAME: HOLY LABS
Ticker: $HOLY

Chain/Standard: ETHERIUM NETWORK

LAUNGUGE: SOLIDITY

The HOLY LABS Platform Is Launched On the EHERIUM Network



HOLY LABS: Deployer

0x7F69b4aD8f97DC1343Ba79E15144932669B7aB78

Audited Files

O.HOLY.sol

Contract Creator Address

0x7F69b4aD8f97DC1343Ba79E15144932669B7aB78

Deployed Contracts:

0x2216848e673541199b9ce168af2b6148f2ad9247

Creator TXH Contracts:

0xe948c1d79b208c337803cfc0941393e75afe369ace293fee5088b4c635711cf8

Auto Contract Scan

Check Report

Share Report

Basic Info

Token Contract Address	0x2216...9247
Owner (Renounced)	0x0000...0000
Total Supply	1B

Risk Check

- cont: Anti whale is No More modifiable
- Reno: Contract Ownership Renounced
- Renounced, Unable to set blacklist
- Doesn't look like honeypot
- Contract is open source
- Owner can not tamper with balance
- Doesn't look like a proxy contract
- Slippage cannot be modified
- Admin privileges abandoned
- Can not Mint
- Can not take back ownership
- No trading-cool-down mechanism

Mechanism Introduction

Buy Tax	3%
Sell Tax	3%

Token Holders Info

Token Holders: 375

Top10 ratio(exclude blackhole)	35.26%
1.0x...3580	99.64M (9.96%)
2.0x...c509	73.09M (7.31%)
3. Uniswapv2: HOLY/WETH	40.83M (4.08%)
4.0x...1c38	33.26M (3.33%)
5.0x...0bcd	33.17M (3.32%)
6.0x...0654	33.16M (3.32%)
7.0x...a41e	21.93M (2.19%)
8.0x...ef39	19.58M (1.96%)
9.0x...9d2c	19.4M (1.94%)
10.0x...e1f6	19.4M (1.94%)

More Details

LP

LP Holders: 2 — Total Supply: 26,457.5131

Percentage of LP locked	100%
1. Blackhole/黑洞地址	26.46K (100%)
2. Blackhole/黑洞地址	0.0{14}1 (0%)

More Details

Sell detection

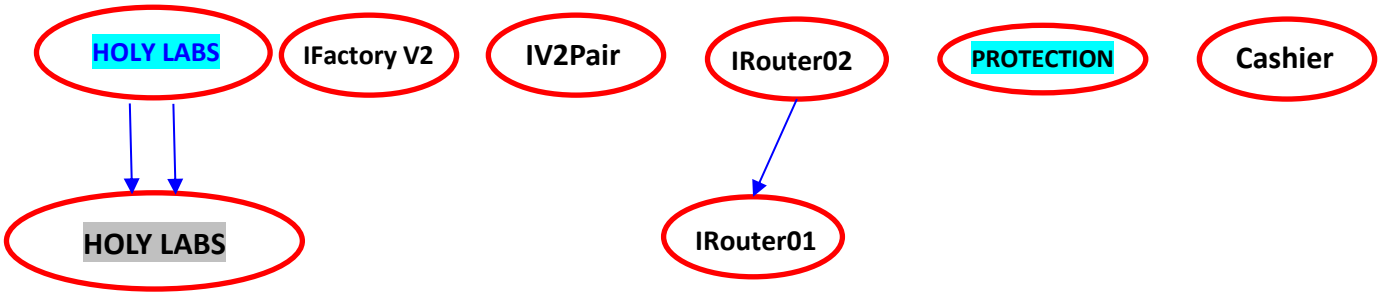
Wallets	Success	Failed	Siphoned
203	203	0	0

Tax	Ave Tax 2.9183%	
	Tax 2%	Count 201
	Tax 12%	Count 1
	Tax 0%	Count 1

Attention

Is anti whale

INHERITANCE GRAPH



Identifier	Definition	Severity
CEN-12	Centralization privileges of HOLY LABS	Medium

Vulnerability 0 : No important security issue detected.
Threat level: Low

```
    function transferOwnership(address newOwner) public virtual onlyOwner {
        require(newOwner != address(0), "Ownable: new owner is the zero address");
        _transferOwnership(newOwner);
    }

    /**
     * @dev Transfers ownership of the contract to a new account (`newOwner`).
     * Internal function without access restriction.
     */
    function _transferOwnership(address newOwner) internal virtual {
        address oldOwner = _owner;
        _owner = newOwner;
        emit OwnershipTransferred(oldOwner, newOwner);
    }
}
```

ISSUES CHECKING STATUS

	Issue Description	Checking Status
1.	Compiler errors.	PASSED
2.	Race Conditions and reentrancy. Cross-Function Race Conditions.	PASSED
3.	Possible Delay In Data Delivery.	PASSED
4.	Oracle calls.	PASSED
5.	Front Running.	PASSED
6.	Sol Dependency.	PASSED
7.	Integer Overflow And Underflow.	PASSED
8.	DoS with Revert.	PASSED
9.	Dos With Block Gas Limit.	PASSED
10.	Methods execution permissions.	PASSED
11.	Economy Model of the contract.	PASSED
12.	The Impact Of Exchange Rate On the solidity Logic.	PASSED
13.	Private use data leaks.	PASSED
14.	Malicious Event log.	PASSED
15.	Scoping and Declarations.	PASSED
16.	Uninitialized storage pointers.	PASSED
17.	Arithmetic accuracy.	PASSED
18.	Design Logic.	PASSED
19.	Cross-Function race Conditions	PASSED
20.	Save Upon solidity contract Implementation and Usage.	PASSED
21.	Fallback Function Security	PASSED

AUDIT RESULT

PASSED

Identifier	Definition	Severity
CEN-02	Initial asset distribution	Minor

All of the initially minted assets are sent to the contract deployer when deploying the contract. This is Normal for most deployer and/or contract owner...

Additional Observations

```
abstract contract Ownable is Context {
    address private _owner;

    event OwnershipTransferred(address indexed previousOwner, address indexed newOwner);

    /**
     * @dev Initializes the contract setting the deployer as the initial owner.
     */
    constructor() {
        _transferOwnership(_msgSender());
    }

    /**
     * @dev Throws if called by any account other than the owner.
     */
    modifier onlyOwner() {
        _checkOwner();
        _;
    }

    /**
     * @dev Returns the address of the current owner.
     */
    function owner() public view virtual returns (address) {
```

RECOMMENDATION

Project stakeholders should be consulted during the initial asset distribution process.

RECOMMENDATION

Deployer and/or contract owner private keys are secured carefully.

Please refer to PAGE-7 CENTRALIZED PRIVILEGES for a detailed understanding.

ALLEVIATION

The HOLY LABS project team understands the centralization risk. Some functions are provided privileged access to ensure a good runtime behavior in the project

References

1 MITRE. CWE-1041: Use of Redundant Code. https://cwe.mitre.org/data/definitions/1041.html.

2 MITRE. CWE-1099: Inconsistent Naming Conventions for Identifiers. https://cwe.mitre.org/data/definitions/1099.html.

3 MITRE. CWE-561: Dead Code. https://cwe.mitre.org/data/definitions/561.html.

4 MITRE. CWE-563: Assignment to Variable without Use. https://cwe.mitre.org/data/definitions/563.html.

5 MITRE. CWE-663: Use of a Non-reentrant Function in a Concurrent Context. https://cwe.mitre.org/data/definitions/663.html.

6 MITRE. CWE-837: Improper Enforcement of a Single, Unique Action. https://cwe.mitre.org/data/definitions/837.html.

7 MITRE. CWE-841: Improper Enforcement of Behavioral Workflow. https://cwe.mitre.org/data/definitions/841.html.

8 MITRE. CWE CATEGORY: Bad Coding Practices. https://cwe.mitre.org/data/definitions/1006.html.

9 MITRE. CWE CATEGORY: Business Logic Errors. https://cwe.mitre.org/data/definitions/840.html.

10 MITRE. CWE CATEGORY: Concurrency. https://cwe.mitre.org/data/definitions/557.html.

11 MITRE. CWE VIEW: Development Concepts. https://cwe.mitre.org/data/definitions/699.html.

12 OWASP. Risk Rating Methodology. https://www.owasp.org/index.php/OWASP_Risk_Rating_Methodology.

Identifier	Definition	Severity
COD-10	Third Party Dependencies	Minor

Smart contract is interacting with third party protocols e.g., Pancakeswap router, cashier contract, protections contract. The scope of the audit treats third party entities as black boxes and assumes their functional correctness. However, in the real world, third parties can be compromised, and exploited. Moreover, upgrades in third parties can create severe impacts, e.g., increased transactional fees, deprecation of previous routers, etc.

RECOMMENDATION

Inspect and validate third party dependencies regularly, and mitigate severe impacts whenever necessary.

DISCLAIMERS

Vital Block Security provides the easy-to-understand audit of Solidity, Move and Raw source codes (commonly known as smart contracts).

The smart contract for this particular audit was analyzed for common contract vulnerabilities, and centralization exploits. This audit report makes no statements or warranties on the security of the code. This audit report does not provide any warranty or guarantee regarding the absolute bug-free nature of the smart contract analyzed, nor do they provide any indication of the client's business, business model or legal compliance. This audit report does not extend to the compiler layer, any other areas beyond the programming language, or other programming aspects that could present security risks. Cryptographic tokens are emergent technologies, they carry high levels of technical risks and uncertainty. You agree that your access and/or use, including but not limited to any services, reports, and materials, will be at your sole risk on an as-is, where-is, and as-available basis. This audit report could include false positives, false negatives, and other unpredictable results.

CONFIDENTIALITY

This report is subject to the terms and conditions (including without limitations, description of services, confidentiality, disclaimer and limitation of liability) outlined in the scope of the audit provided to the client. This report should not be transmitted, disclosed, referred to, or relied upon by any individual for any purpose without InterFi Network's prior written consent.

NO FINANCIAL ADVICE

This audit report does not indicate the endorsement of any particular project or team, nor guarantees its security. No third party should rely on the reports in any way, including to make any decisions to buy or sell a product, service or any other asset. The information provided in this report does not constitute investment advice, financial advice, trading advice, or any other sort of advice and you should not treat any of the report's content as such. This audit report should not be used in any way

to make decisions around investment or involvement. This report in no way provides investment advice, nor should be leveraged as investment advice of any sort.

FOR AVOIDANCE OF DOUBT, SERVICES, INCLUDING ANY ASSOCIATED AUDIT REPORTS OR MATERIALS, SHALL NOT BE CONSIDERED OR RELIED UPON AS ANY FORM OF FINANCIAL, TAX, LEGAL, REGULATORY, OR OTHER ADVICE.

TECHNICAL DISCLAIMER

ALL SERVICES, AUDIT REPORTS, SMART CONTRACT AUDITS, OTHER MATERIALS, OR ANY PRODUCTS OR RESULTS OF THE USE THEREOF ARE PROVIDED "AS IS" AND "AS AVAILABLE" AND WITH ALL FAULTS AND DEFECTS WITHOUT WARRANTY OF ANY KIND. TO THE MAXIMUM EXTENT PERMITTED UNDER APPLICABLE LAW, VITAL BLOCK HEREBY DISCLAIMS ALL WARRANTIES, WHETHER EXPRESSED, IMPLIED, STATUTORY, OR OTHERWISE WITH RESPECT TO SERVICES, AUDIT REPORT, OR OTHER MATERIALS. WITHOUT LIMITING THE FOREGOING, VITAL BLOCK SPECIFICALLY DISCLAIMS ALL IMPLIED WARRANTIES OF MERCHANTABILITY, FITNESS FOR A PARTICULAR PURPOSE, TITLE AND NON-INFRINGEMENT, AND ALL WARRANTIES ARISING FROM THE COURSE OF DEALING, USAGE, OR TRADE PRACTICE.

WITHOUT LIMITING THE FOREGOING, VITAL BLOCK MAKES NO WARRANTY OF ANY KIND THAT ALL SERVICES, AUDIT REPORTS, SMART CONTRACT AUDITS, OR OTHER MATERIALS, OR ANY PRODUCTS OR RESULTS OF THE USE THEREOF, WILL MEET THE CLIENT'S OR ANY OTHER INDIVIDUAL'S REQUIREMENTS, ACHIEVE ANY INTENDED RESULT, BE COMPATIBLE OR WORK WITH ANY SOFTWARE, SYSTEM, OR OTHER SERVICES, OR BE SECURE, ACCURATE, COMPLETE, FREE OF HARMFUL CODE, OR ERROR-FREE.

TIMELINESS OF CONTENT

The content contained in this audit report is subject to change without any prior notice. Vital Block does not guarantee or warrant the accuracy, timeliness, or completeness of any report you access using the internet or other means, and assumes no obligation to update any information following the publication.

LINKS TO OTHER WEBSITES

This audit report provides, through hypertext or other computer links, access to websites and social accounts operated by individuals other than Vital Block. Such hyperlinks are provided for your reference and convenience only and are the exclusive responsibility of such websites and social accounts owners. You agree that Vital block Security is not responsible for the content or operation of such websites and social accounts and that Vital Block shall have no liability to you or any other person or entity for the use of third-party websites and social accounts. You are solely responsible for determining the extent to which you may use any content at any other websites and social accounts to which you link from the report.

CERTIFICATE BY VITAL BLOCK SECURITY



VITALBlock SKYNET

ABOUT VITAL BLOCK

Vital Block provides intelligent blockchain Security Solutions. We provide solidity and Raw Code Review, testing, and auditing services. We have Partnered with 15+ Crypto Launchpads, audited 50+ smart contracts, and analyzed 200,000+ code lines. We have worked on major public blockchains e.g., Ethereum, Binance, Cronos, Doge, Polygon, Avalanche, Metis, Fantom, Bitcoin Cash, Aptos, Oasis, etc.

Vital Block is Dedicated to Making Defi & Web3 A Safer Place. We are Powered by Security engineers, developers, UI experts, and blockchain enthusiasts. Our team currently consists of 5 core members, and 8+ casual contributors.



1000+
Audits Completed



$12B+
Secured



1M+
Lines of Code Audited

@vb_audit

@Vitalblock

@vitalblock

github.com/VBS-LABS

Info@vitalblock.org

www.vitalblock.org





Blockchain Security | Smart Contract Audit | KYC Certification | SAFU .

MADE IN CANADA